Python for computational Problem Solving · Unit_02 · Unit 2: Collections & Functions

DRAFT — UE26CS151A · Unit_02 · Unit 2: Collections & Functions · Files

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · Files
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

Files

1 Files

Study focus: Files. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000046 · EV0000047

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Command line arguments.

Key details

- My main intention is that I want to perform addition of two numbers addition of two numbers using command line arguments.
- And this is my Y, but my argument in this case could be two.
- Sum is equal to X plus Y.
- And I want to previously let me print the values how it was initially.
- Anyway, I don't need them.
- I will be doing it.
- I want to find out what is the initial value and how will I give the how will I get the final result.
- Fine, and when I do this, just look at what happens.
- It says index list index out of range.
- The reason is when I am passing an input as a command line arguments, my input also should be a part of the command line itself.
- This is R g of zero, this is R g of one, and R V of two.
- My final output I'm expecting is a fifty in this particular case.
- We have successfully executed giving an input through the command line arguments.
- Previously you would have already seen the other way of giving inputs.
- You want it to be an integer, so you say it as an int of inputs.
- I will give an input called as twenty, the other one is thirty again.
- The inputs that we are trying to give is very minimal.
- humanly impossible to specify hundred different values on the command line.

- The reason may be we may have some errors while giving in the sense I may end up just giving 99 or I may end up giving one out one also.

Example or application from the lecture

- So take a simple example in this case.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000046 · EV0000047 · EV0000048 · +1 source units in pack.json

Core explanation · 05:00–09:59

But in both the cases it is not feasible.

Key details

- there may be a situation where you want to work with a particular application where the amount of data that is required is more for you.
- The larger data is required.
- Hopefully it is clear to you why we need files.
- Data is persistent, even after your program terminates, the data is persistent.
- You're going to be working with larger amount of data.
- A file is some information or data which stays in a computer storage device, especially in computer memory.
- One is called as a binary, and the other one is called as a text file.
- Binary files, it contains the sequence.
- And very difficult for human to read.
- At the same time, when you have a text file, you can store data in the form of alphabets, digits and other special symbols.
- In this session we are going to be working with the text files.
- When I say we will be working with the files, a file may be used for a reading, or it can be used for writing.
- May be used for reading or it can be used for writing purpose.
- When I see a file, I can use an existing file, or I can create a new file, or I can overwrite an existing file.
- You can make use of an existing file, so overwrite, both means the same in this case, or you can create your own files.
- Reading and writing.
- You may also specify reading as R.
- Writing as W, reading as R and writing as W.
- Where R and W represent modes.
- R and W represents modes.
- There are other modes also, but they are out of scope for our syllabus, so we are going to be discussing only a read and write operation.
- One is open a file.
- Reading or writing operation.
- At the same time you will also have to close the file.

- You will also have to close the file.
- Hope it is clear.
- If a file that I want to open or before that, in order to open a file, I will make use of a function called as open.
- I want to open it only for the reading purpose.
- Now in this case if t2.txt is in existence, which means if it is present, then open function will return an object.
- Open function will return an object for this particular.

Example or application from the lecture

- And at the same time, it's chances of errors are very less when input is given through a particular file.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000050 · EV0000051 · EV0000052 · +1 source units in pack.json

Core explanation · 10:00–14:59

F is a file object.

Key details

- On success, so it gives me if t2.txt does not exist, it throws me an error.
- Let me repeat.
- If one success, it returns a file object.
- If it is not present, it throws me an error.
- It's still executing.
- Maybe I haven't commented on it.
- We don't need them.
- In this case, what am I trying to do?
- I'm trying to open a particular file.
- If it is not present, it throws us in.
- File not found error has been thrown.
- When you create a file, make sure your dot txt and dot p y resides in a same part.
- resides in a same path.
- Otherwise you will have to specify the entire path where your text file is available.
- In and both my.py files and both.txt files are present on the desktop at this particular point.
- What is that output?
- And it says it is success because twin.txt was already available for us, so and I was successfully able to open the text file.
- This is how we try to open a particular file.
- There are different ways to perform a read operation.
- One is the first one is by making use of something called as read line.
- There is something called as a read line.
- The other one is read lines.
- And the last one is actually read.

- Read comes with and without parameter.
- Read comes with and without parameters.
- This is Python files dot txt and dot py should be in the same location.
- This is what the contents of my text sir.
- Yes, T1.txt file.
- Let me execute again.
- I do not want this.
- in read mode.
- We are asking you to open.
- I want you to open it in a read mode.
- And I want to read.

Example or application from the lecture

- Let's not worry about it.
- Let's not worry.
- Let's go back here and we will perform these operations on that particular text file.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000054 · EV0000055 · EV0000056 · +1 source units in pack.json

Core explanation · 15:00–19:59

If you can see it has read the first line from the file, but there is a huge gap between this and our prompt.

Key details

- Is this fine?
- The reason is the function read line, it reads a line from a file, in the sense read line is reading this from a file along with reading, it also includes a new line character.
- It also includes a new line characters at the end of the line in a file.
- Along with that.
- Along with that, you do have a print statement.
- By default, print statement already has a new line within it, because of which I am getting two lines as an extra line here.
- No, we do not want any extra lines to be available.
- I need to get rid of this particular line.
- read line dot strip.
- Make sure you assign it to the same variable.
- A sign it to the same variable.
- Okay, if you can see we have got rid of one line from this to this because print statement already has a new line because of which we are getting this particular line.
- If you still want to get rid of that also, what you can do is you can specify.
- And is equal to nothing.
- We will not have that extra line also in the output.

- So similarly we will go to the next one and I will just copy paste a few things so that I want to make my life easy.
- In this case I will use read.
- Read is a function which reads your entire file.
- Read is a function which basically reads your entire file.
- And see the entire file has been read.
- Or if you are interested, you're also free to read only certain bytes of information from your particular file.
- This is also possible.
- At the same time, you can also read the entire file.
- There are other ways of reading a file where you can iterate over using your while loop and then get each line using your read line and print the same.
- any extra lines you can make use of your strip function and get rid of a particular line.
- Okay, so the next one is we went through read line, we also had a read option and there is one more which is called as read lines.
- I will use these two lines.
- I don't want the previous one, so I will comment it.
- A read lines reads the whole file into a list.

Example or application from the lecture

- Let's look into this now and find out the difference.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000058 · EV0000059 · EV0000060 · +1 source units in pack.json

Core explanation · 20:00–24:59

Please remember read file reads the entire or a whole file into a list.

Key details

- Where each element of the list is a line of a file.
- Where each element of a list is a line of a particular file.
- Print read lines.
- These are the few things that we have learned.
- Okay next one is to perform writing operation to a file.
- write to a file.
- Some file where we will say or I will make use of the same file t1.txt itself I will make use.
- But I have to open it in a write mode.
- The moment you open it in a write mode, your previous content will be lost.
- If you can remember my previous contents were this is Python files, dot txt and dot py should be in the same location.
- What we will do is we will try to print.
- To which one?
- To a file.
- Even.txt whose handle is F.

- If you remember, I just want to show it to you.
- When we did your print statement.
- When we did your print statement.
- File is equal to system dot standard output.
- You wanted your output to be available.
- All this time your output was available on the standard output.
- But right now, whatever you are writing, you want your output to be available in a particular file, which is t win.txt.
- File is equal to F.
- Exits I'll say python files.py and it has successfully executed.
- We had opened it long back, so I just closed the file and then opened it for viewing the content.
- we strictly tell you to just make use of a print statement in order to write it to a file.
- Hope you have understood the concept of files.

Example or application from the lecture

- In the notes, we have also given you using a write statement, okay, write as a function.
- So in the coming sessions we will also try to work with a few example programs.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000062 · EV0000063 · EV0000064 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Files transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/046_Files__6b7a6928ed4ec68a.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Files transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/046_Files__6b7a6928ed4ec68a.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Files transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/046_Files__6b7a6928ed4ec68a.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9998; only the JSON segmentation supplies the displayed timestamps.
Files transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/046_Files__6b7a6928ed4ec68a.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. No historical archive question page met the unit-match candidate threshold; all subject-level and ambiguous pages remain in datasets/question-archive.json.
2. 3 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00010	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0063. Sidecar variants compare...
SRC00011	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0063. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00012	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0063. Sidecar variants compare...
SRC00013	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0063. Sidecar variants compare...

C Revision and retrieval practice